
UNIT 10 HASHING

Structure

10.0	Introduction
10.1	Objectives
10.2	Drivers and motivations for hashing
10.3	Index Mapping
10.3.1	Challenges with Index Mapping
10.3.2	Hash Function
10.3.3	Simple Hash Example
10.4	Collision Resolution
10.4.1	Separate Chaining
10.4.2	Open Addressing
10.4.3	Double Hashing
10.5	Comparison of Collision Resolution Methods
10.6	Load Factor and Rehashing
10.6.1	Rehashing
10.7	Summary
10.8	Solutions/Answers
10.9	Further Readings

10.0 INTRODUCTION

Hashing is a key technique in information retrieval. Hashing transforms the input data into a small set of keys that can be efficiently stored and retrieved. Hashing provides constant time and highly efficient information retrieval capability irrespective of total search space.

10.1 OBJECTIVES

After going through this unit, you should be able to

- the drivers and motivations for hashing,
- various hashing techniques,
- understand index mapping
- collision avoiding techniques,
- understand load factor and rehashing

10.2 MAIN DRIVERS AND MOTIVATIONS FOR HASHING

As part of searching and information retrieval we use hashing for mainly below given reasons:

- Provide constant time data retrieval and insertion
- Manage the data related to large set of input keys efficiently
- Provide cost efficient hash key computations

10.3 INDEX MAPPING

In order to store and retrieve huge set of data, we can think of using a large sized array and store/retrieve the data from the large array. We can plan to use the data value as the key for the array. As the array indexing takes only $O(1)$ time, the method guarantees a constant and fast performance.

Index mapping or trivial hashing method assumes a large sized array and the input keys as index for the array to retrieve the value.

Let's look at an example for this index mapping method. Let's design a large array based on the index mapping method. We use the array to store the name of the user and we plan to lookup the user details using their 4-digit employee id. As depicted in Figure 1, we create a large array to accommodate all the employee ids and use the employee id as an index to get the user details from the array.

Index mapping approach can be used when we know or when we can predict all the input keys. For instance, if we were to store the details for months of a year we know that we can have maximum of 12 months and hence we can design the hash table with size 12. Similarly, for input keys such as days of month, countries, states within a country where we can predict the maximum values, we can use the index mapping method.

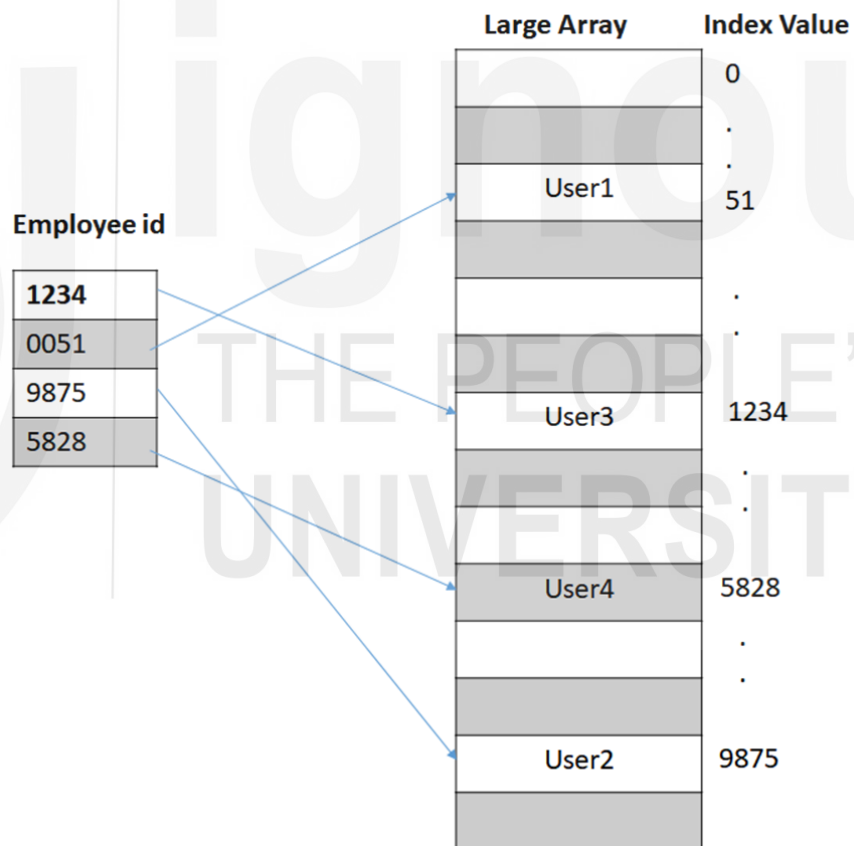


Figure 1 Index mapping approach

10.3.1 Challenges with Index Mapping

We can observe two main drawbacks with the index mapping approach. Firstly, this approach needs only non-negative integer values as the index keys and secondly as we can see from Figure 1, the array size poses performance and scalability challenges. The array has to be sized to accommodate the largest key value and the values are unevenly distributed leading to wastage of space.

To overcome the two limitations mentioned above, we need a conversion function that converts all data types (string, image, alphanumeric etc.) into a non-negative integer

value. Additionally, the conversion function also maps the input values to smaller set of keys that can be used as index for a smaller sized array. We call the conversion function as “hash function”.

Let’s use a hash function that takes the input values (employee id) and converts the value into a key that forms of the index for the hash table storing the employee details. We have used the below given hash function:

$$h(x) = x \bmod 10$$

where $h(x)$ is the hash value, x is the input key. The mod operation provides the remainder value for the key. As a result of this hash function we now have the hash table with optimal size as depicted in Figure 2.

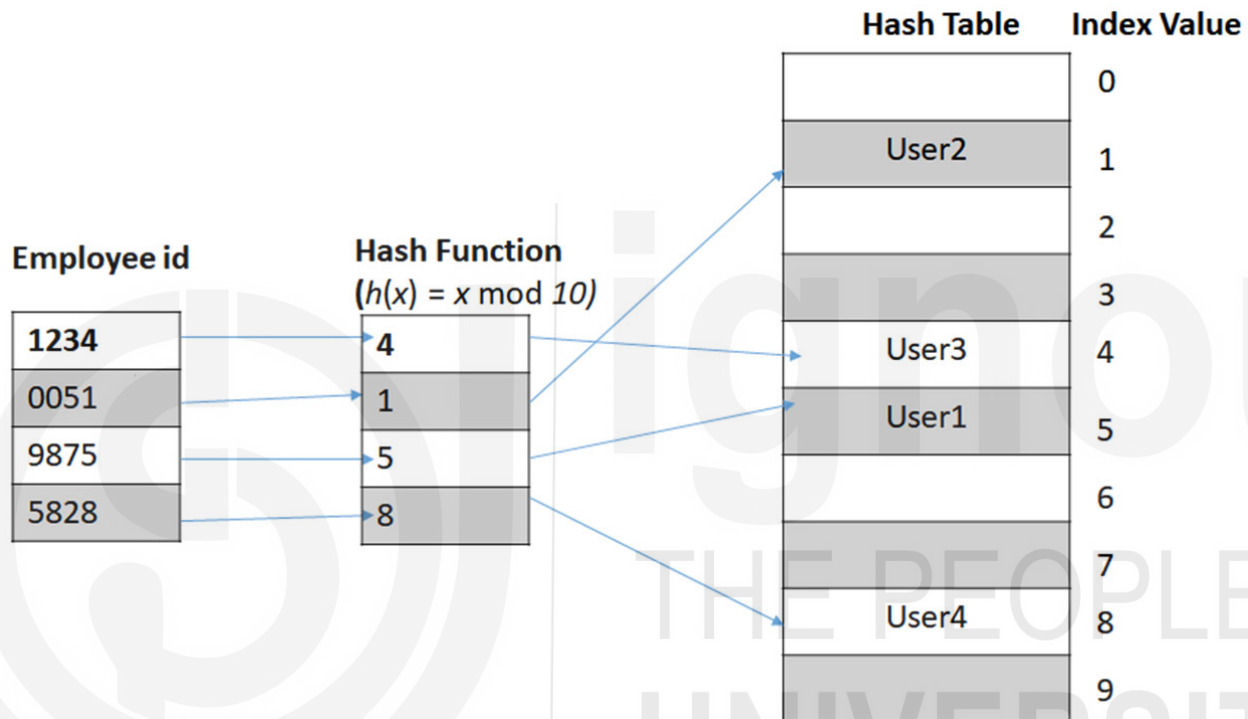


Figure 2 Hash Function and Hash Table

As we can see from Figure 2, instead of creating a huge hash table of size 9875, we have now managed to store the elements within an array of size 10. The hash function has converted the input into smaller set of keys that are used as index for the hash table.

Let us re-look at two challenges we saw in our earlier direct access method. The examples we discussed in Figure 2 use integer values as input keys. If we use non-integer values such as images or strings, we need to convert it first into a non-negative integer value. For instance, using the corresponding ascii values for each of its character, we can get a numeric value for a string. We can then use the hash function to create a fingerprint for the numeric value to store the corresponding details in the right sized hash table.

10.3.2 Hash Function

As we saw in the previous example, a hash function reduces a large non-negative integer into a smaller hash value which is used as an index into the hash table for searching the value.

The efficiency of a hash function is determined by following characteristics:

- **Computing efficiency** – The hash function should compute the hash value quickly and efficiently even for large key values
- **Uniform distribution** – The hash function should be able to distribute the keys evenly in the hash table.
- **Deterministic** – The hash function should consistently create the same key for a given value.
- **Minimal Collisions** – The hash function should minimize the key collisions in the hash table.

10.3.3 Simple Hash Example

Let us look at implementation of main functions using a modulo-based hash function.

We assume no collisions for this sample code in Java.

// the function returns the stored value for the input key

```
public V getValue(int inputKey) {  
    int hashvalue = getHashValue(inputKey);  
    return this.hashArray[hashvalue].value;  
}
```

//the function adds the value for the given input key

```
public void addValue(int inputKey V inputValue) {  
    int hashvalue = getHashValue(inputKey);  
    this.hashArray[hashvalue].value = inputValue ;  
}
```

//the function computes the hash value using mod logic

```
public int getHashValue(int inputKey) {  
    return inputKey % this.hashArray.length;  
}
```

//the function removes the input value from the hashArray

```
public void removeValue(int inputKey) {  
    int hashvalue = getHashValue(inputKey);  
    this.hashArray[hashvalue].value = null;  
}
```

10.4 COLLISION RESOLUTION

For large set of input keys, we might end up having same hash value for two different input values. For instance, let us consider our simple hash function $h(x) = x \bmod 10$

As the hash function provides the remainder value, if we have two input keys 24 and 4874, the hash value will be 4. These cases cause collision as both the input keys 24 and 4874 compete for same slot in the hash table.

We discuss three key collision resolution techniques in subsequent sections.

10.4.1 Separate Chaining

In separate chaining method, we chain the values to the same spot in the hash table. We use a data structure like linked list that can store multiple values.

The example Figure 3 depicts collision handling using the modulo based hash function. The input values 0051 and 821 result in the same hash value of 1. In the hash table we chain the values for user 2 and user 5 for the same slot.

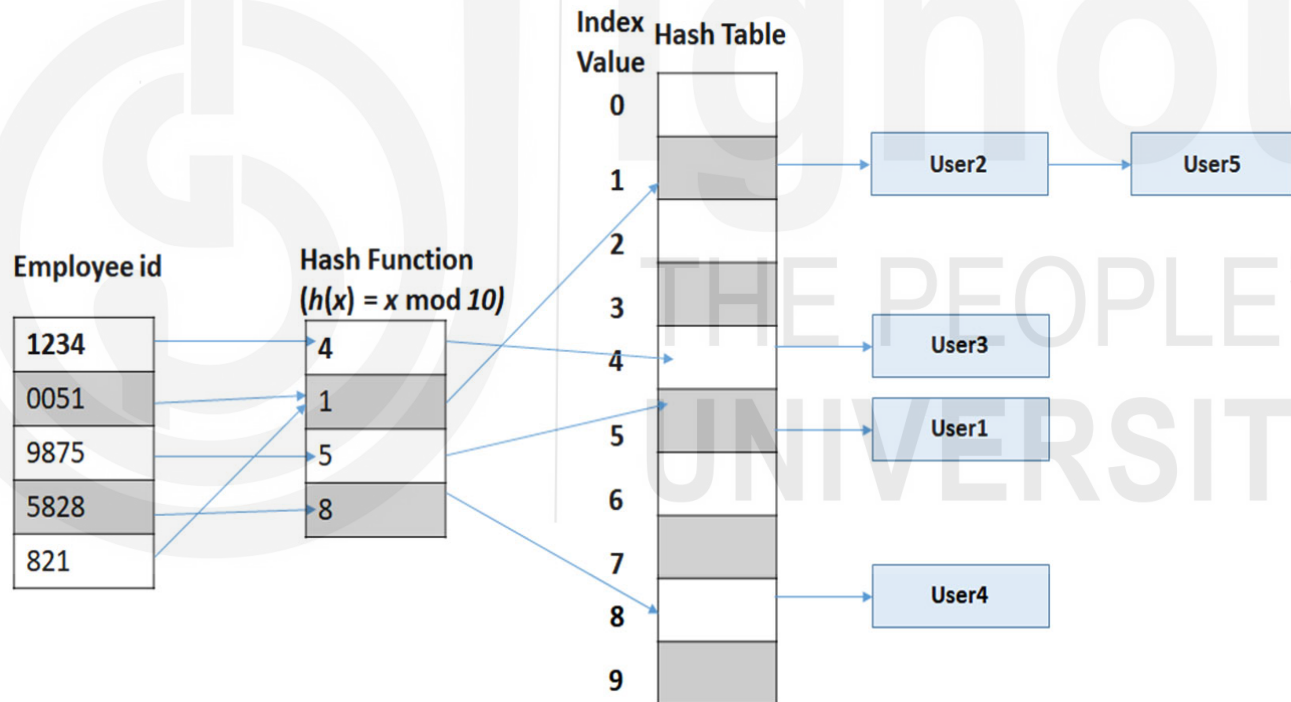


Figure 3 Separate Chaining

We are chaining the data values user2 and user5 for the slot 0 in the hash table.

Insert Operation

Given below are the steps for the insert operation using separate chaining method to insert a key k:

1. Compute the hash value of the key k using the hash function.
2. Use the computed hash value as the index in the hash table to find the slot for the key k.
3. Add the key k to the linked list at the slot.

Search operation

Given below are the steps for the search operation for key k:

1. Compute the hash value of the key k using the hash function.
2. Use the computed hash value as the index in the hash table to find the slot for searching the key k.
3. Check for all elements in the linked list at the slot for a match with key k.

Check Your Progress – 1

1. _____ provides direct mapping of keys to the hash table index.
2. The two key challenges with index mapping are _____.
3. The main characteristics of a hash function are _____.
4. The time complexity of index mapping is _____.
5. _____ data structure can be used to chain multiple values to the same spot in the hash table.
6. Hash value should always be non-negative. True/False

10.4.2 Open Addressing

In the open addressing collision addressing strategy, we search for the next available slot in the hash table when the natural slot is not available. We search for the open or unused locations in the hash table.

There are mainly two variants of open addressing – linear probing and quadratic probing. In the linear probing we sequentially iterate to the next available spot.

We define the linear probe by the following equation for i^{th} iteration:

$$h(x, i) = (h(x) + i) \bmod m \text{ where } m \text{ is the hash table size}$$

We need to accordingly change the other hash functions such as getHashValue, putHashValue and deleteHashValue.

- For getHashValue we need to start from the initial slot (hashTable[h(x)]) we need to check the subsequent slots till we find the matching key.
- For putHashValue we need to start with the initial slot (hashTable[h(x)]) till we find an empty slot or till the hashtable is full.
- For deleteHashValue we can place null into the slot or use an availability marker (occupied, available).

The linear probing leads to a situation known as “primary clustering” wherein the consecutive slots form “cluster” of keys in the hash table. As the cluster size grows, it impacts the efficiency of the probing for placing the next key.

Quadratic probing makes larger jumps to avoid the primary clustering. We define the quadratic probe by the following equation for i^{th} iteration:

$$h(x, i) = (h(x) + i^2) \bmod m$$

As we can see in the equation, for every iteration the quadratic probing makes larger jumps. In Quadratic probing we encounter “secondary clustering” problem.

Let us look at an example for linear probing to avoid the collision. Let us consider the following set of keys [56, 1072, 97, 84, 60] and the hash table size of 5. When we apply mod based hash function and start placing the keys in the appropriate slots we get the placement as depicted in Figure 4.

Index Value	Hash Table
0	60
1	56
2	1072
3	97
4	84

Figure 4 Linear Probing

The value 56 goes to position 1 due to the mod value of $(56 \bmod 5)$ operation. Similarly, 1072 assumes position 2. However, when we try to place the next element 97 we end up with a collision at slot 2. So, we find the next empty slot at slot 3 and place 97 there. Rest of the elements 84 and 60 go to the positions 4 and 0 respectively based on their mod values.

10.4.3 Double Hashing

We can avoid the challenges with primary clustering and secondary clustering using the double hashing strategy. Double hashing uses a second hash function to resolve the collisions. The second hash function is different from the primary hash function and uses the key to yield non-zero value.

The first hash function in the double hashing finds the initial slot for the key and the second hash function determines the size of jumps for the probe. The i^{th} probe is defined as follows

$$h(x, i) = (h_1(x) + i * h_2(x)) \bmod m \text{ where } m \text{ is the hash table size}$$

Let us look at an example for the double hashing to understand it better. Let us consider a hash table of size 5 and we use the below given hash functions:

$$H_1(x) = x \bmod 5$$

$$H_2(x) = x \bmod 7$$

Let us try to insert two elements 60 and 80 into the hash table. We can place the first element 60 at slot 0 based on the hash function. When we try to insert the second element 80, we face a collision at slot 0. For the first iteration we apply the double hashing as follows:

$$H(80, 1) = (0 + 1 * 3) \bmod 5 = 3$$

Hence, we now place the element 80 in slot 3 to avoid collision as depicted in figure 5.

Index Value	Hash Table
0	60
1	
2	
3	80
4	

Figure 5 Double Hashing Example

10.5 COMPARISON OF COLLISION RESOLUTION METHODS

Comparison of linear probing, quadratic probing and double hashing is given below:

Collision Resolution Technique	Separate Chaining	Open Addressing - Linear Probing	Open Addressing - Quadratic Probing	Double Hashing
Primary clustering	No	Yes	No	No
Secondary clustering	No	No	Yes	No
Key storage in Hash table	Inside & Outside Hashtable	Inside Hash table	Inside Hash table	Inside Hash table

10.6 LOAD FACTOR AND REHASHING

The hash table provides constant time complexity for operations such as retrieval, insertion and deletion with lesser keys. As the key size grows, we run out of vacant spots in the hash table leading to collision that impacts the time complexity. When the collision happens, we need to re-adjust the hash table size so that we can accommodate additional keys. Load factor defines the threshold when we should re-size the hash table to main the constant time complexity.

Load factor is the ratio of the elements in the hash table to the total size of the hash table. We define load factor as follows:

Load factor = (Number of keys stored in the hash table)/Total size of the hash table.

In open addressing as all the keys are stored within the hash table, the load factor is ≤ 1 . In separate chaining method as the keys can be stored outside the hash table, there is a possibility of load factor exceeding the value of 1.

If the load factor is 0.75 then as soon as the hash table reaches 75% of its size, we increase its capacity. For instance, lets consider the hash table of size 10 with load factor of 0.75. We can insert seven hash keys into this hash table without triggering the re-size. As soon as we add the eighth key, the load factor becomes 0.80 that

exceeds the configured threshold triggering the hash table resize. We normally double the hash table size during the resize operation.

10.6.1 Rehashing

when the load factor exceeds the configured value, we increase the size of hash table. Once we do it we should also re-compute the hash values for the existing keys as the size of the hash table has changed. This process is called “rehashing”. Rehashing is a costly exercise especially if the key size is huge. Hence it is necessary to carefully select the optimal initial size of the hash table to start with.

Given below are the high-level steps for rehashing:

1. For each new key insert into the hash table, compute the load factor
2. If the load factor exceeds the pre-defined value then increase the hash table size (normally we double the hash table size)
3. Recompute the hash value (rehash) for each of the existing elements in the hash table.

Let us look at the rehashing with an example. We have a hash table of size 4 with load factor of 0.60. Let's start by inserting these elements – 30, 31 and 32. We can insert 30 at slot 2 and 31 at slot 3 and 32 at slot 0. Insertion of 32 triggers the hash table resize as the load factor has breached the threshold of 0.60. As a result, we double the hash table size to 8.

With the new hash table size, we need to recalculate the hash values of the already inserted keys. Key 30 will now be placed in slot 6, key 31 will be placed in slot 7 and key 32 in slot 0.

Check Your Progress – 2

1. The main techniques of open addressing are _____
2. Load factor triggers _____
3. Linear probing leads to _____ clustering
4. _____ probing make large jumps leading to secondary clustering
5. Second hash function in double hashing can result in 0. True/False
6. _____ should be done for the existing keys of the hash table post resizing.

10.7 SUMMARY

In this unit, we started discussing the main motivations for the hashing. Hashing allows us to store and retrieve large data efficiently.

Index mapping uses the input values as direct index into the hash table. Index mapping requires huge hash table size leading to inefficiencies. When we handle large size input values we encounter collision where multiple input values compete for the same spot in the hash table. The main collision resolution techniques are separate chaining and open addressing. In separate chaining we chain the values that get mapped to a spot. We use linear probing and quadratic probing as part of open addressing technique to find the next available spot. We use two hash functions as part of double hashing. Load factor determines the trigger for the hash table resizing and once the hash table is resized, we re-compute the hash values of the existing keys using rehashing.

10.8 SOLUTIONS/ANSWERS

Check Your Progress – 1

1. Index mapping
2. handling non integer keys and large hash table size
3. computing efficiency, uniform distribution, deterministic and minimal collisions
4. $O(1)$
5. Linked List
6. True

Check Your Progress – 2

1. linear probing, quadratic probing and double hashing
2. resizing of hash table
3. primary
4. quadratic
5. False
6. Rehashing

10.9 FURTHER READINGS

Horowitz, Ellis, Sartaj Sahni, and Susan Anderson-Freed. *Fundamentals of data structures*. Vol. 20. Potomac, MD: Computer science press, 1976.

Cormen, Thomas H., et al. *Introduction to algorithms*. MIT press, 2022.

Lafore, Robert. *Data structures and algorithms in Java*. Sams publishing, 2017.

Karumanchi, Narasimha. *Data structures and algorithms made easy: data structure and algorithmic puzzles*. Narasimha Karumanchi, 2011.

West, Douglas Brent. *Introduction to graph theory*. Vol. 2. Upper Saddle River: Prentice hall, 2001.

https://en.wikipedia.org/wiki/Hash_function#Trivial_hash_function

https://en.wikibooks.org/wiki/A-level_Computing/AQA/Paper_1/Fundamentals_of_data_structures/Hash_tables_and_hashing

<https://ieeexplore.ieee.org/book/8039591>